```java
package com.example.demo.payment.command;
import com.example.demo.payment.entity.PaymentMethod;
import com.example.demo.payment.entity.PaymentType;

import lombok.Data;
import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
@Data
public class PaymentCommand {

        private PaymentType paymentType;

        private String referenceId;

        private String projectId;

        private double amount;

        private PaymentMethod paymentMethod;

        private String transactionId;

        private String remarks;

        private String createdBy;


}
```

```java
package com.example.demo.payment.command;
import com.example.demo.payment.entity.PaymentMethod;

import lombok.Data;
import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
@Data
public class PaymentUpadateCommand {
    private double amount;

    private PaymentMethod paymentMethod;

    private String transactionId;

    private String remarks;

}
```

```java
package com.example.demo.payment.controller;

import com.example.demo.payment.command.PaymentCommand;
import com.example.demo.payment.command.PaymentUpadateCommand;
import com.example.demo.payment.DTO.PaymentDTO;
import com.example.demo.payment.entity.PaymentType;
import com.example.demo.payment.service.PaymentService;
import lombok.RequiredArgsConstructor;
import org.springframework.web.bind.annotation.*;

import java.time.LocalDate;
import java.util.List;

@RestController
@RequestMapping("/api/payments")
@RequiredArgsConstructor
public class PaymentController {

    private final PaymentService paymentService;

    @PostMapping
    public PaymentDTO createPayment(@RequestBody PaymentCommand command) {

        return paymentService.createPayment(command);
    }

    @GetMapping("/{id}")
    public PaymentDTO getPayment(@PathVariable String id) {

        return paymentService.getPayment(id);
    }

    @GetMapping("/project/{projectId}")
    public List<PaymentDTO> getByProject(@PathVariable String projectId) {

        return paymentService.getByProject(projectId);
    }

    @GetMapping("/reference/{referenceId}")
    public List<PaymentDTO> getByReference(@PathVariable String referenceId) {

        return paymentService.getByReference(referenceId);
    }

    @PutMapping("/{id}")
    public PaymentDTO updatePayment(@PathVariable String id,
                                    @RequestBody PaymentUpadateCommand command) {

        return paymentService.updatePayment(id, command);
    }

    @PutMapping("/cancel/{id}")
    public PaymentDTO cancelPayment(@PathVariable String id) {

        return paymentService.cancelPayment(id);
    }

    // Filter payments
    @GetMapping("/filter")
    public List<PaymentDTO> getPayments(
```

```
            @RequestParam PaymentType paymentType,
            @RequestParam(required = false) String referenceId,
            @RequestParam LocalDate startDate,
            @RequestParam LocalDate endDate) {

        return paymentService.getPayments(
                paymentType,
                referenceId,
                startDate,
                endDate
        );
}
}
```

```
package com.example.demo.payment.DTO;

import com.example.demo.payment.entity.*;
import lombok.*;

import java.time.LocalDateTime;

@Getter
@Setter
@AllArgsConstructor
@NoArgsConstructor
@Builder
public class PaymentDTO {

    private String paymentId;

    private PaymentType paymentType;

    private String referenceId;

    private String projectId;

    private double amount;

    private PaymentMethod paymentMethod;

    private String transactionId;

    private LocalDateTime paymentDate;

    private PaymentStatus status;

    private String remarks;

    public static PaymentDTO fromEntity(Payment payment) {

        return PaymentDTO.builder()
```

```
                .paymentId(payment.getPaymentId())
                .paymentType(payment.getPaymentType())
                .referenceId(payment.getReferenceId())
                .projectId(payment.getProjectId())
                .amount(payment.getAmount())
                .paymentMethod(payment.getPaymentMethod())
                .transactionId(payment.getTransactionId())
                .paymentDate(payment.getPaymentDate())
                .status(payment.getStatus())
                .remarks(payment.getRemarks())
                .build();
    }
}
```

```
package com.example.demo.payment.entity;
import com.example.demo.payment.entity.PaymentMethod;
import com.example.demo.payment.entity.PaymentStatus;
import com.example.demo.payment.entity.PaymentType;
import  lombok.Getter;
import lombok.Setter;
import lombok.*;
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;
import java.time.LocalDate;
import java.time.LocalDateTime;
@Document(collection = "payment")
@Getter
@Setter
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class Payment {

    @Id
    private String paymentId;

    private PaymentType paymentType;

    private String referenceId;

    private String projectId;

    private double amount;

    private PaymentMethod paymentMethod;

    private String transactionId;
```

```
    private LocalDateTime paymentDate;

    private PaymentStatus status;

    private String remarks;

    private String createdBy;

    private LocalDateTime createdAt;

    private LocalDateTime updatedAt;

}
```

```
package com.example.demo.payment.entity;

public enum PaymentMethod {
    CASH,
    UPI,
    BANK_TRANSFER,
    CHEQUE,
    CARD
}
```

```
package com.example.demo.payment.entity;

public enum PaymentStatus {
    PENDING,
    PAID,
    FAILED,
    CANCELLED
}
```

```
package com.example.demo.payment.entity;

public enum PaymentType {
    SALARY,
    MATERIAL,
    EXPENSE,
    VENDOR
}
```

```java
package com.example.demo.payment.repository;

import com.example.demo.payment.entity.Payment;
import com.example.demo.payment.entity.PaymentType;
import org.springframework.data.mongodb.repository.MongoRepository;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.List;

public interface PaymentRepository extends MongoRepository<Payment, String> {

    List<Payment> findByProjectId(String projectId);

    List<Payment> findByReferenceId(String referenceId);

    List<Payment> findByPaymentTypeAndPaymentDateBetween(
            PaymentType paymentType,
            LocalDateTime startDate,
            LocalDateTime endDate
    );

    List<Payment> findByPaymentTypeAndReferenceIdAndPaymentDateBetween(
            PaymentType paymentType,
            String referenceId,
            LocalDateTime startDate,
            LocalDateTime endDate
    );

}
```

```java
package com.example.demo.payment.service;

import com.example.demo.payment.command.PaymentCommand;
import com.example.demo.payment.command.PaymentUpadateCommand;
import com.example.demo.payment.DTO.PaymentDTO;
import com.example.demo.payment.entity.*;
import com.example.demo.payment.repository.PaymentRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.List;
import java.util.stream.Collectors;

@Service
@RequiredArgsConstructor
public class PaymentService {
```

```java
private final PaymentRepository paymentRepository;

public PaymentDTO createPayment(PaymentCommand command) {

    Payment payment = Payment.builder()
            .paymentType(command.getPaymentType())
            .referenceId(command.getReferenceId())
            .projectId(command.getProjectId())
            .amount(command.getAmount())
            .paymentMethod(command.getPaymentMethod())
            .transactionId(command.getTransactionId())
            .remarks(command.getRemarks())
            .createdBy(command.getCreatedBy())
            .paymentDate(LocalDateTime.now())
            .status(PaymentStatus.PAID)
            .createdAt(LocalDateTime.now())
            .build();

    paymentRepository.save(payment);

    return PaymentDTO.fromEntity(payment);
}

public PaymentDTO getPayment(String id) {

    Payment payment = paymentRepository.findById(id)
            .orElseThrow(() -> new RuntimeException("Payment not found"));

    return PaymentDTO.fromEntity(payment);
}

public List<PaymentDTO> getByProject(String projectId) {

    return paymentRepository.findByProjectId(projectId)
            .stream()
            .map(PaymentDTO::fromEntity)
            .collect(Collectors.toList());
}

public List<PaymentDTO> getByReference(String referenceId) {

    return paymentRepository.findByReferenceId(referenceId)
            .stream()
            .map(PaymentDTO::fromEntity)
            .collect(Collectors.toList());
}

public PaymentDTO updatePayment(String id, PaymentUpadateCommand command) {

    Payment payment = paymentRepository.findById(id)
            .orElseThrow(() -> new RuntimeException("Payment not found"));

    payment.setAmount(command.getAmount());
    payment.setPaymentMethod(command.getPaymentMethod());
    payment.setTransactionId(command.getTransactionId());
    payment.setRemarks(command.getRemarks());
    payment.setUpdatedAt(LocalDateTime.now());

    paymentRepository.save(payment);

    return PaymentDTO.fromEntity(payment);
```

```java
    }

    public PaymentDTO cancelPayment(String id) {

        Payment payment = paymentRepository.findById(id)
                .orElseThrow(() -> new RuntimeException("Payment not found"));

        payment.setStatus(PaymentStatus.CANCELLED);
        payment.setUpdatedAt(LocalDateTime.now());

        paymentRepository.save(payment);

        return PaymentDTO.fromEntity(payment);
    }

    public List<PaymentDTO> getPayments(
            PaymentType paymentType,
            String referenceId,
            LocalDate startDate,
            LocalDate endDate) {

        LocalDateTime start = startDate.atStartOfDay();
        LocalDateTime end = endDate.atTime(23, 59, 59);

        List<Payment> payments;

        if (referenceId == null || referenceId.isEmpty()) {

            payments = paymentRepository
                    .findByPaymentTypeAndPaymentDateBetween(
                            paymentType,
                            start,
                            end
                    );

        } else {

            payments = paymentRepository
                    .findByPaymentTypeAndReferenceIdAndPaymentDateBetween(
                            paymentType,
                            referenceId,
                            start,
                            end
                    );
        }

        return payments.stream()
                .map(PaymentDTO::fromEntity)
                .collect(Collectors.toList());
    }
}
```

```java
package com.example.demo.framework.config;

import com.example.demo.framework.security.JwtAuthenticationFilter;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.authentication.AuthenticationManager;
import
org.springframework.security.config.annotation.authentication.configuration.Authenticatio
nConfiguration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import
org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    private final JwtAuthenticationFilter jwtAuthenticationFilter;

    public SecurityConfig(JwtAuthenticationFilter jwtAuthenticationFilter) {
        this.jwtAuthenticationFilter = jwtAuthenticationFilter;
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {

        http
                .csrf(csrf -> csrf.disable())
                .sessionManagement(session ->
                        session.sessionCreationPolicy(SessionCreationPolicy.STATELESS)
                )
                .authorizeHttpRequests(auth -> auth
                        .requestMatchers(
                                "/api/user/register/{role}",

                                "/api/project/lookup/All",
                                "/api/BuildingDetails/view",
                              "/api/payroll/**",
                              "/api/user/login",
//                                "/api/attendance/checkin/{userId}",
//                                "/api/attendance/checkout/{userId}",


                                "/swagger-ui/**",
                                "/swagger-ui.html",
                                "/v3/api-docs/**"
                        ).permitAll()
//  USER CONTROLLER

.requestMatchers("/api/user/findby/{id}").hasAnyRole("SUPERADMIN","CLIENT","ADMIN")

.requestMatchers("/api/user/deleteby/{id}").hasAnyRole("SUPERADMIN","CLIENT","ADMIN")

.requestMatchers("/api/user/lookup/all").hasAnyRole("SUPERADMIN","ADMIN")
```

```
//  PROJECT CONTROLLER
                                .requestMatchers("/api/project/create").hasRole("ADMIN")

.requestMatchers("/api/project/lookupBy/{projectId}").hasAnyRole("ADMIN","SUPERADMIN","CL
IENT","CIVILENGINEER","CONTRACTOR","ARCHITECT")

.requestMatchers("/api/project/update/{projectId}").hasAnyRole("CIVILENGINEER","ADMIN","S
UPERADMIN")

.requestMatchers("/api/project/deleteBy/{projectID}").hasAnyRole("SUPERADMIN","CLIENT","A
DMIN","CIVILENGINEER","CONTRACTOR")
// ATTENDANCE

                                // ☑ 1️⃣ Check-In
                                .requestMatchers(HttpMethod.POST,
"/api/attendance/checkin/**")
                                .hasAnyRole("ADMIN", "CIVILENGINEER", "LABOR",
"CONTRACTOR")

                                // ☑ 2️⃣ Check-Out
                                .requestMatchers(HttpMethod.POST,
"/api/attendance/checkout/**")
                                .hasAnyRole("ADMIN", "CIVILENGINEER", "LABOR",
"CONTRACTOR")

                                // ☑ 3️⃣ Attendance History
                                .requestMatchers(HttpMethod.GET,
"/api/attendance/get/History/**")
                                .hasAnyRole("ADMIN", "CIVILENGINEER", "LABOR",
"CONTRACTOR")

                                // ☑ 4️⃣ Get All Attendance By Date (ADMIN Only)
                                .requestMatchers(HttpMethod.GET,
"/api/attendance/all/date/**")
                                .hasRole("ADMIN")

                                // ☑ 5️⃣ Attendance Report
                                .requestMatchers(HttpMethod.GET,
"/api/attendance/report")
                                .hasAnyRole("ADMIN", "CONTRACTOR")


//  TODAY WORK CONTROLLER
                                .requestMatchers("/civil/today-
work/add").hasAnyRole("CIVILENGINEER","ADMIN")
                                .requestMatchers("/civil/today-work/today/by-
date").hasAnyRole("CONTRACTOR","ADMIN","SUPERADMIN")
                                .requestMatchers("/civil/today-
work/view_work").hasAnyRole("CIVILENGINEER","CLIENT","CONTRACTOR","ADMIN","SUPERADMIN")
                                .requestMatchers("/civil/today-
work/updateTodayWork/{projectName}").hasAnyRole("CIVILENGINEER","ADMIN")
                                .requestMatchers("/civil/today-
work/deleteTodayWork/{projectName}/{workId}").hasAnyRole("CIVILENGINEER","ADMIN")

//  LABOR ATTENDANCE CONTROLLER
                                .requestMatchers("/api/Labor/Attendance/mark-
attendance").hasAnyRole("CIVILENGINEER","ADMIN")
                                .requestMatchers("/api/Labor/Attendance/update-
attendance").hasAnyRole("CIVILENGINEER","ADMIN")

//  PROJECT DAILY REPORT CONTROLLER
```

```
.requestMatchers("/ProjectDaily/Report").hasAnyRole("CIVILENGINEER","ADMIN")

.requestMatchers("/ProjectDaily/Report/**").hasAnyRole("ADMIN","CONTRACTOR")

.requestMatchers("/ProjectDaily/View_workReport").hasRole("ADMIN")

.requestMatchers("/ProjectDaily/create_progressReport").hasAnyRole("CONTRACTOR","ADMIN")

.requestMatchers("/ProjectDaily/view_all").hasAnyRole("CIVILENGINEER","CLIENT","CONTRACTO
R","ADMIN","SUPERADMIN")

.requestMatchers(HttpMethod.PUT,"/ProjectDaily/updateProgressReport/**").hasAnyRole("ADMI
N","CONTRACTOR")

//  MATERIAL CONTROLLER

.requestMatchers("/api/material/**").hasAnyRole("CIVILENGINEER","ADMIN")

.requestMatchers("/api/material/MaterialReport").hasAnyRole("CIVILENGINEER","ADMIN")

.requestMatchers("/api/material/view_MaterialReport").hasRole("ADMIN")

.requestMatchers("/api/material/UpdateMaterialReport/{materialReportId}").hasAnyRole("ADM
IN","CIVILENGINEER")

//  BUILDING CONTROLLER

.requestMatchers("/api/buildings/**").hasAnyRole("CIVILENGINEER","ADMIN","SUPERADMIN")

.requestMatchers("/api/buildings/Update/{buildingId}").hasAnyRole("CIVILENGINEER","ADMIN"
,"SUPERADMIN")

.requestMatchers("/api/buildings/lookup/{buildingId}").hasAnyRole("CIVILENGINEER","ADMIN"
,"SUPERADMIN")
                                .requestMatchers("/api/buildings/add-
flat").hasAnyRole("CIVILENGINEER","ADMIN","SUPERADMIN")

//.requestMatchers(HttpMethod.DELETE,"/api/flats/{flatId}").hasAnyRole("CIVILENGINEER","A
DMIN","SUPERADMIN")

//.requestMatchers(HttpMethod.DELETE,"/api/room/{flatId}").hasAnyRole("CIVILENGINEER","AD
MIN","SUPERADMIN")

//  WORK DETAILS CONTROLLER

.requestMatchers("/api/workDetails/add").hasAnyRole("CIVILENGINEER","ADMIN","SUPERADMIN")

.requestMatchers("/api/workDetails/view").hasAnyRole("CIVILENGINEER","ADMIN","SUPERADMIN"
)

.requestMatchers("/api/workDetails/update/{workId}").hasAnyRole("CIVILENGINEER","ADMIN")

//  BILLING CONTROLLER

.requestMatchers("/api/billing/**").hasAnyRole("CIVILENGINEER","ADMIN","SUPERADMIN")

//  PAYMENT CONTROLLER

                        // CREATE PAYMENT
                        .requestMatchers(HttpMethod.POST, "/api/payments")
                        .hasAnyRole("ADMIN", "CONTRACTOR")
```

```
                        // UPDATE PAYMENT
                        .requestMatchers(HttpMethod.PUT, "/api/payments/{id}")
                        .hasRole("ADMIN")

                        // CANCEL PAYMENT
                        .requestMatchers(HttpMethod.PUT, "/api/payments/cancel/**")
                        .hasRole("ADMIN")

                        // VIEW PAYMENTS
                        .requestMatchers(HttpMethod.GET, "/api/payments/**")
                        .hasAnyRole(
                                "ADMIN",
                                "CONTRACTOR",
                                "CIVILENGINEER",
                                "ARCHITECT",
                                "CLIENT")

                        // for filter
                                .requestMatchers(HttpMethod.GET, "/api/payments/filter"
                                ).hasAnyRole("ADMIN","CONTRACTOR")

////
.requestMatchers("/api/material/addPlumbingMaterials").hasRole("CIVILENGINEER")
////
.requestMatchers("/api/material/addFinishingMaterials").hasRole("CIVILENGINEER")
////
.requestMatchers("/api/material/addElectricalMaterials").hasRole("CIVILENGINEER")
////                        .requestMatchers("/api/material/allMateril-
report").hasRole("CIVILENGINEER")
////
.requestMatchers("/api/material/view/MaterialRemaining").hasAnyRole("CIVILENGINEER","ADMI
N","SUPERADMIN")

////
.requestMatchers("/api/billing/all").hasAnyRole("CIVILENGINEER","ADMIN","SUPERADMIN")
////
.requestMatchers("/api/billing/find/{clientId}").hasAnyRole("CIVILENGINEER","ADMIN","SUPE
RADMIN")
////
.requestMatchers("/api/billing/delete/{clientId}").hasAnyRole("CIVILENGINEER","ADMIN","SU
PERADMIN")
////
.requestMatchers("/api/billing/Expense/create}").hasAnyRole("CIVILENGINEER","ADMIN","SUPE
RADMIN")
////
.requestMatchers("/api/billing/Expense/all}").hasAnyRole("CIVILENGINEER","ADMIN","SUPERAD
MIN")
////
.requestMatchers("/api/billing/Expense/{id}}").hasAnyRole("CIVILENGINEER","ADMIN","SUPERA
DMIN")

                        .anyRequest().authenticated()
                )
                .addFilterBefore(
                        jwtAuthenticationFilter,
                        UsernamePasswordAuthenticationFilter.class
                );

        return http.build();
    }
```

```java
    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public AuthenticationManager authenticationManager(
            AuthenticationConfiguration configuration
    ) throws Exception {
        return configuration.getAuthenticationManager();
    }
}
```